ChatGPT

‘Exploring Gen AI’ 深度研究：结构化中文总结与分析报告

执行摘要

“Exploring Gen AI”在 **MartinFowler.com** 上呈现为一个持续更新的“备忘录（memos）合集入口页”，旨在记录 Thoughtworks [1] 技术人员对生成式 AI 在软件开发中的实践探索，而非一篇单独、定稿的长文。 [2]
该系列整体立场并非“AI 会自动替代开发者”，而是强调：GenAI 的价值与风险都很强，关键在于把它放进可控的工程系统（测试、约束、反馈回路、安全与审查机制）里使用，并持续迭代这些“护栏/马具（harness）”。 [3]
系列作者反复指出：LLM 的非确定性意味着我们永远只能谈概率与风险管理；“把人彻底移出回路”的“vibe coding”只适合低风险、易发现错误的场景，高影响场景需要更强的可观察性、验证与治理。 [4]
对工程团队而言，最可落地的结论是：把 AI 当作“产生候选变更的加速器”，而把确定性的质量门禁（测试、静态分析、结构约束、最小权限、审计日志）当作“决定能否合入/发布的系统”。 [5]

语料范围与研究方法

入口页《Exploring Generative AI》（用户称“Exploring Gen AI”）本质上是一个系列索引：它解释了写作动机，并按时间列出大量备忘录链接；页面署名为 Birgitta Böckeler [6]（并非由用户所说的 Martin Fowler 单独撰写），且明确为“Thoughtworks technologists 的探索记录”。 [2]
因此，本报告在“结构映射”部分以该索引页的组织方式为骨架（入口页叙述 + 按时间推进的备忘录群），并选取其中对“AI 辅助编码工作流、质量与风险治理”最关键的代表性备忘录进行归纳、引用与对比（覆盖 2023–2026 的重要节点，如：内联补全、TDD、multi-file editing、agentic 工具链、供应链安全、上下文工程、harness 工程、人-代理回路）。 [7]
需要强调：该系列大量内容是作者/团队的实践观察与“工程判断”，而非严格对照实验；文中出现的百分比与时间估计也多为经验性估算，应理解为“启发式信号”，而非可直接外推的行业统计。 [8]

原文结构映射与关键摘录

> 说明：入口页自身几乎不含分节小标题，其“结构”主要体现为（a）开篇动机说明，（b）按时间顺序的备忘录列表。为满足“映射到标题/子标题”的要求，下文以“入口页叙述”与“按年份阶段的备忘录群（相当于索引页的子条目集合）”呈现，并在每个阶段给出中文释义与 2–3 条关键英文引文（含中文译文）。 [2]

入口页叙述：为何要“探索”而不是下结论

入口页将生成式 AI 的出现描述为“信息浪潮”：它既诱人也不确定，因此需要持续记录试验与观点演化，而不是要么忽视、要么给出过早的统一结论。 [2]

关键引文：
1) English: “The release of ChatGPT and other websites using OpenAI's GPT models has sparked massive

interest in Large Language Models (LLMs)."
中文： "ChatGPT 以及其他基于 OpenAI GPT 模型的网站发布，引发了人们对大型语言模型（LLMs）的巨大兴趣。" [2]

2) English: "One way to deal with this wave of information is to simply ignore it. But that doesn't work well if you want to establish a useful point of view."
中文："面对这波信息浪潮，一种方式是干脆忽略它；但如果你想形成有用的观点，这就行不通。" [2]

3) English: "We post these explorations, as we do them, in short memos."
中文："我们在探索进行的同时，以简短备忘录的形式发布记录。" [2]

早期备忘录阶段：从"工具版图"到"何时有用、何时伤害"（2023）

这一阶段建立了三个底层直觉：
其一，先画出工具链全景（任务类型、交互方式、提示词构成、模型属性、托管方式），避免被单点产品宣传牵着走。 [9]
其二，内联补全/IDE 辅助并非总是提效；它的有效性与技术栈流行度、问题类型、建议粒度、开发者经验、容错空间等高度相关。 [10]
其三，风险不仅是"写错代码"，还包括"把坏实践放大""审查疲劳/自动化偏见导致的草率合入"，以及被生成结果"锚定"而难以重新思考方案。 [11]

关键引文：
1) English: "Whenever there is a new area with still evolving patterns and technology, I try to develop a mental model of how things fit together."
中文："当一个新领域的模式与技术仍在演化时，我会尝试建立一个关于'各部分如何拼在一起'的心智模型。" [9]

2) English: "The tool cannot distinguish good code from bad code."
中文："工具无法分辨好代码与坏代码。" [12]

3) English: "Sometimes it's ok to take a break from the assistant."
中文："有时候暂时关掉助手、休息一下是可以的。" [12]

工程实践阶段：把测试与反馈回路放到更核心的位置（2023）

在讨论 GitHub Copilot [13] 时，系列提出一个"反直觉但工程上更稳"的观点：AI 并不会让测试过时，反而让测试（尤其是快速、准确的单元测试反馈）更重要，因为你不仅要验证自己写的代码，也要验证 AI 写的代码。 [14]
同时，系列明确反对把 coding assistant 等同于结对编程：结对的价值不仅是"更快写代码"，还包括团队流动、集体代码所有权、隐性知识传递与协作技能，而这些很难由 LLM 替代。 [15]

关键引文：
1) English: "TDD is especially needed when using AI coding assistants."
中文："在使用 AI 编码助手时，TDD 尤其必要。" [14]

2) English: "Starting with a blank test file doesn't mean starting with a blank context."
中文："从空白测试文件开始，并不意味着上下文也是空白。" [14]

3) English: “Use coding assistants to make pairs better, not to replace pairing.”
中文：“用编码助手让结对更强，而不是替代结对。” [15]

扩展边界阶段：从“代码片段”走向“跨文件变更/代理”（2024）

到 2024，系列的关注点从“几行/一个函数的补全”明显转向“更大变更面”：例如“多文件编辑（multi-file editing）”让 AI 一次会话可以改动多个文件，带来工作流的“步进变化”，但也显著扩大 blast radius（影响半径），并提高人类漏检风险与提交过大带来的发布风险。 [16]
同一时期，作者用真实项目（如 Bahmni [17] / OpenMRS [18] 的“遗留代码库”）展示：AI 在“定位相关代码簇、解释领域术语”上可能有帮助，但在缺乏可运行环境、缺乏良好测试与模块化的遗留系统里，AI 也会像人一样被卡住；AI 不能替代自动化、文档与可复现的开发环境。 [19]

关键引文：
1) English: “This represents a step change for coding assistance workflows.”
中文：“这代表了编码辅助工作流的一次‘台阶式变化’。” [20]

2) English: “Wait! I’d estimate it took 30-60 seconds, for the tasks I tried it with.”
中文：“等一下！以我试过的任务来看，大约需要 30–60 秒。” [20]

3) English: “AI cannot magically replace a well-documented and well-automated setup.”
中文：“AI 不能‘魔法般’替代一个文档完善且高度自动化的环境搭建。” [19]

风险与质量阶段：从“能跑”到“能长期维护、能安全交付”（2025）

2025 年的备忘录把关注点拉回到“工程负债与安全”：
一方面，作者指出代理式工具在“监督式协作”里能极大提速，但依然需要资深工程判断持续介入，否则会在“时间到提交”“迭代团队流动”“长期可维护性”三个半径上制造隐形成本（如重复实现、冗余测试、过度复杂、误诊问题）。 [21]
另一方面，代理式 coding assistants 增加了软件供应链攻击面（尤其当代理通过工具调用、Model Context Protocol [22] 服务器、规则文件等与外部系统交互时），因此需要 sandbox、最小权限、审计与威胁建模把“新链路”纳入既有安全工程。 [23]

关键引文：
1) English: “Always carefully review AI-generated code. It’s very rare that I do NOT find something to fix or improve.”
中文：“永远要认真审查 AI 生成的代码；我几乎总能发现需要修正或改进的点。” [24]

2) English: “Without careful oversight, though, the AI agents seem to have a strong tendency to introduce technical debt.”
中文：“如果缺少细致监督，AI 代理似乎很容易引入技术债。” [25]

3) English: “Coding assistants can undeniably provide a boost. However, the enhanced capabilities come with significantly expanded security implications.”
中文：“编码助手无疑能带来增益，但能力增强也伴随显著扩大的安全影响。” [26]

方法论收束阶段：风险评估、上下文工程、harness 工程与“人在回路之上”（2025–2026）

这一阶段形成了系列最具“可复用框架”的叙事：
其一，“vibe coding”不是道德判断，而是风险决策问题：概率（会不会错）× 影响（错了多严重）× 可发现性（能否及时发现）。[27]
其二，随着代理工具加入“规则/技能/子代理/钩子/AGENTS.md[28] / MCP”等能力，上下文工程成为核心工程活动，但它并不让结果“确定”，只能改变成功概率；因此应当“渐进式构建上下文”，并警惕“控制幻觉”。[29]
其三，更宏观的定位是：人不必逐行当 gatekeeper（in the loop），也不该完全放手（out of the loop），而应“on the loop”——设计并持续改进工作回路本身（规格、检查、约束、反馈），这组机制就是 agent 的 harness；进一步可构建“agentic flywheel”，让代理协助改进 harness。[30]

关键引文：
1) English: “Risk assessment is typically a combination of three factors: Probability, Impact, Detectability.”
中文：“风险评估通常由三个因素构成：发生概率、影响、可发现性。”[31]

2) English: “Context engineering is curating what the model sees so that you get a better result.”
中文：“上下文工程就是整理模型能看到的内容，从而得到更好的结果。”[32]

3) English: “The right place for us humans is to build and manage the working loop… Let’s call this ‘on the loop.’”
中文：“人类更合适的位置是构建并管理工作回路……把这称为‘on the loop（在回路之上）’。”[33]

核心论证、对 AI 辅助编码的含义与作者建议

主要论证脉络

该系列贯穿始终的主张可以归纳为五条：

第一，GenAI 并非“更高级的代码生成器/编译器”，而是非确定性的“推断器”，因此工程讨论的基本单位应从“正确/错误”转为“概率、风险与反馈回路”。[34]

第二，AI 编码工具越强，越需要把“确定性的质量系统”前移：测试、静态检查、结构约束、可观察性与审计，构成能规模化使用代理的前提（“shift-left”对人有效，对代理也有效）。[35]

第三，“更大范围的自动化”（multi-file edits、异步代理、自动 PR）扩大了 blast radius 与供应链攻击面；因此需要把代理的工具调用（如 MCP）、规则文件与外部依赖纳入威胁建模与最小权限治理。[36]

第四，“人在回路中的位置”应动态可调：低风险/高可发现性可“少审甚至不审”，但高影响/低可发现性场景必须强化审查与验证；成熟团队应把精力投入到“改进 harness”，而非永远逐行盯代码。[37]

第五，AI 的“效率红利”并非免费：它可能以“技术债、重复实现、过度复杂、测试脆弱化、团队流动摩擦”形式回收，因此需要持续监控内部质量与团队体验。[38]

对 AI 辅助编码的直接含义

在该系列框架下，AI 辅助编码更像“多层回路系统”而非单一功能：

- **代码层（最内环）**：用 AI 生成候选实现，但必须有快速反馈（单元测试/类型系统/静态检查）驱动迭代；TDD 被视为提高输入质量、缩短反馈回路、并帮助“分而治之”的关键实践。[39]
- **变更层（中间环）**：当工具能跨文件执行计划时，工程师应把更多时间用于“制定变更计划 + 组织工作集 + 审查 diff + 校验测试金字塔”，并警惕大提交与漏检风险。[16]
- **系统层（外环）**：通过 context engineering 与 harness engineering，把“什么是对的/如何验证/出错如何失败得更响亮”编码进仓库与流水线，让代理能自检、自修、并在必要时升级给人。[40]

系列中较明确的建议清单（来自不同作者备忘录）

1) **把测试当作 AI 时代的“首要反馈机制”**：强调 TDD/单元测试对 AI 生成代码的验证价值，并给出 TDD+Copilot 的具体操作建议（例如在测试文件顶部提供上下文、用 Given-When-Then 改善意图表达、必要时“删除并重生成”）。[14]

2) **不要把 coding assistant 当“结对替代品”**：建议把工具用于“增强结对”，并认为结对能缓解 review fatigue 与质量风险。[41]

3) **持续进行微型风险评估**：用“概率×影响×可发现性”决定审查强度与 AI 使用方式。[27]

4) **上下文工程要“渐进式”并追求透明**：避免一次性堆砌规则文件；工具应提供上下文占用可视化，团队应迭代维护规则/技能/钩子等配置。[42]

5) **安全措施要把代理纳入供应链威胁模型**：建议 sandbox/最小权限、对 MCP/规则文件做供应链审查、加强可观测与审计、避免自动接受变更、把代理工作流纳入 threat modeling。[26]

6) **面对复杂任务要允许“重来/重写”**：在与 Matteo Vaccari [43] 的实践叙事中，当 AI 生成实现“看似合理但并不真正理解”时，作者给出“必要时从头重写”的经验教训。[44]

假设、局限与开放问题

关键假设

该系列多数建议默认一个前提：团队已经具备或愿意建设“工程护栏”（测试、CI、静态分析、结构约束、可观测性），并将其视为在 AI 时代同样甚至更重要的基础设施。[45]
同时，很多论证隐含“代码结构越清晰、边界越明确，AI 越有效”的假设（即所谓“AI-friendly codebase design”/模块化、关注点分离、可导航性）。[46]

明确局限

1) **证据形态以实践观察为主**：作者多次强调某些内容是“轶事/不是研究”，并呼吁更多真实使用报告。[47]
2) **上下文工程存在“控制幻觉”**：即使有规则、技能、工作流与清单，LLM 仍可能不遵守或过度执行；因此“工程化”并不等价于“确定性”。[48]

3) **自动化扩张带来安全新弱点**：MCP、规则文件、工具调用链与代理权限使传统监控更难识别“微妙的数据泄露/指令注入”。 [23]

开放问题（作者明确提出或强烈暗示）

- **推理模型在编码任务中到底有多关键？** 作者在讨论推理模型时明确表示“还没有答案”，并指出上下文质量、无函数调用等限制可能约束推理价值。 [49]
- **Spec-driven development 会不会把“评审负担”从代码转移到大量 markdown 制品？** 作者担心过度产出文档导致“审查过载”，甚至“制造更糟的改进（Verschlimmbesserung）”。 [50]
- **harness 是否会像服务模板一样“分叉与同步困难”？** 作者在谈 harness 作为“未来服务模板”时直接提出这种演化/治理疑问。 [51]
- **长期尺度下的“全代理生成代码库”如何保持架构一致性？** 相关讨论在引用 OpenAI [52] 的实践中被明确标注为仍在学习的问题。 [53]

与行业主流立场的对比与对齐点

行业主流产品叙事的共同点

GitHub [54] 在 Copilot 页面强调其覆盖“编辑器、终端、以及可将 issue 分配给 coding agents 的工作流”，并将其定位为能“autonomously write code, create pull requests, and respond to feedback”。 [55]
Google [56] 对 Jules [57] 的官方介绍强调“异步、代理式、在云 VM 中克隆仓库执行任务、交付 plan/reasoning/diff”，并突出“能处理复杂、多文件变更”。 [58]
Anthropic [59] 的 Claude Code [60] 文档则把重点放在“阅读代码库、跨文件编辑、运行命令、集成开发工具”，并提供 hooks/skills 等增强可控性的机制。 [61]
此外，Model Context Protocol [22] 作为开放协议被定位为连接 LLM 应用与外部工具/数据源的标准化方式。 [62]

与本系列观点的差异与互补

1) **营销强调“自治与提速”，系列强调“回路工程与治理成本”**
行业叙事更多突出“代理能做更多”，而系列强调：能力提升会把瓶颈推向审查、QA、安全与结构一致性；若不先投资 harness/反馈回路，自治只会扩大风险半径。 [63]

2) **系列更明确地将“风险管理”制度化**
系列把“概率×影响×可发现性”作为决定审查强度与是否可 vibe 的框架，并把“人在回路之上（on the loop）”作为可扩展的组织策略；这比多数产品页更接近“工程运营手册”。 [64]

3) **在“工具扩展性”上，系列与行业方向高度对齐**
系列对规则/技能/钩子/MCP/AGENTS.md 的讨论，与主流产品正在强化的“可配置、可集成、可治理”的路线一致，只是系列更强调其非确定性边界与安全代价。 [65]

工程团队采用 GenAI 辅助编码的可执行清单

简表：系列反复出现的“收益—风险—对策”模式

模式/用法（系列常见）	主要收益	主要风险	建议的工程对策（系列给出的方向）
IDE 内联补全/小片段生成	减少查资料与重复敲代码的摩擦，维持 flow [66]	放大坏实践、上下文被污染、审查疲劳/自动化偏见 [12]	控制建议粒度；定期“关掉休息”；保持批判性审查；结对/代码审查分担认知负担 [67]
多文件编辑 / 大改动会话	降低跨文件机械改动的认知负担 [16]	blast radius 增大、人类漏检、提交过大导致发布与回滚风险 [20]	先“计划再执行”；限制工作集文件；以测试变更作为审查入口点；强化反馈回路 [68]
异步代理/PR 级自治	把时间从“敲代码”转向“定义意图、复核结果” [69]	供应链攻击面上升、权限升级/数据泄露、自动接受导致审批疲劳 [26]	sandbox/最小权限；审计日志；威胁建模纳入代理链路；禁止默认 auto-accept [26]
harness/上下文工程（rules/skills/hooks/MCP）	提升成功概率、让代理能自检自修、减少人类瓶颈 [70]	控制幻觉、配置过载、文档“腐烂”、成本上升 [71]	渐进式构建；可视化上下文占用；把关键约束编码为确定性检查（lint/structural tests） [72]

Mermaid 流程图：基于系列思想的“采用与使用决策流程”

（该流程将系列中“风险评估三因子”“on the loop/harness 思路”“测试与安全门禁”合并为可执行决策树。） [73]

```
flowchart TD
  A[需求/变更出现] --> B[做微型风险评估：概率×影响×可发现性]
  B --> C{低影响 且 高可发现性?}
  C -- 是 --> D[低审查强度：可小范围“vibe”<br/>但仍运行基础自动检查]
  C -- 否 --> E{高影响 或 低可发现性?}
  E -- 是 --> F[高审查强度：人保持在回路附近<br/>明确验证策略]
  E -- 否 --> G[中等审查强度：限定工作集/分步骤迭代]

  D --> H[选择交互方式：内联/多文件编辑/异步代理]
  F --> H
  G --> H

  H --> I[准备上下文：必要文件 + 规则/技能/说明文档<br/>保持上下文精简]
  I --> J[让代理/助手产出候选变更]
  J --> K[确定性门禁：测试/静态分析/结构约束/格式化]
  K --> L{门禁通过?}
  L -- 否 --> M[迭代：修复或收缩任务范围<br/>必要时重启会话/重写]
  M --> J
  L -- 是 --> N[安全门禁：最小权限、审计、依赖/MCP检查、威胁建模对齐]
```

```
  N --> O[提交PR/评审]
  O --> P[上线后监控：错误/性能/业务指标]
  P --> Q[on the loop：把经验回写进harness<br />规则/钩子/测试/结构约束/文档]
  Q --> A
```

小型图表：系列中出现的“主观估计型数据”可视化

下图仅将作者在不同备忘录中的**经验性估计**取中位数做示意：
- “编码助手有用的场景比例”被描述为约 40–60%；
- “一次建议能把你推进到的完成度”约 40–80%；
- “监督式代理模式在某个约 15K LOC 代码库中对作者‘有帮助的比例’约 80%”。 [74]

```
xychart-beta
  title "系列中的经验估计（示意：取区间中位数）"
  x-axis ["有用场景占比(40-60%)","推进完成度(40-80%)","监督式代理有用占比(~80%)"]
  y-axis "百分比(%)" 0 --> 100
  bar [50,60,80]
```

5–7 项行动清单（面向工程师/团队）

1) **先把“反馈回路”做硬**：确保本地/CI 中有快速、可信的测试与静态检查；把它们当作 AI 生成代码的“真相来源”。 [75]

2) **把 AI 使用从“写代码”迁移到“写意图 + 写验证”**：在高风险任务里，优先让人定义可验证的规格/测试/验收标准，再让代理生成实现。 [76]

3) **每次变更先做三因子风险评估**（概率×影响×可发现性），据此决定：是否可 vibe、是否需要结对、是否要拆小、是否要限制多文件编辑工作集。 [77]

4) **渐进式建设项目级“上下文资产”**：用规则/技能/说明文档（例如 AGENTS.md/类似文件）记录实际可执行的构建、测试、风格与安全注意事项；避免一次性堆砌导致“控制幻觉”。 [78]

5) **把“重复犯错”升级为“系统性护栏”**：当你发现代理常犯某类错，不要只在对话里纠正；要把纠错写进 harness（更好的上下文、更严格的 lint/结构测试、更清晰的质量门禁）。 [79]

6) **把代理纳入供应链与权限模型**：默认最小权限、可审计、可回滚；审查 MCP 服务器/规则文件/依赖变更；明确禁止“默认自动接受”。 [80]

7) **保留“必要时重来”的权力**：当会话变得混乱或代码质量不可控时，重启会话、收缩任务范围、甚至从头重写，往往比“被锚定在错误方案上继续修补”更省。 [81]

原文与主要一手来源链接（入口页 + 文中引用的核心主源优先）

说明：按用户要求提供可复制链接（以代码块形式列出）。其中包含入口页与若干关键备忘录，以及系列明确引用/讨论的一手资料（官方博客、协议规范、论文等）。 [82]

```
入口页（系列索引）
https://martinfowler.com/articles/exploring-gen-ai.html

关键备忘录（与工作流/治理最相关）
https://martinfowler.com/articles/exploring-gen-ai/humans-and-agents.html
https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html
https://martinfowler.com/articles/exploring-gen-ai/context-engineering-coding-
agents.html
https://martinfowler.com/articles/exploring-gen-ai/to-vibe-or-not-vibe.html
https://martinfowler.com/articles/exploring-gen-ai/06-tdd-with-coding-
assistance.html
https://martinfowler.com/articles/exploring-gen-ai/software-supply-chain-attack-
surface.html
https://martinfowler.com/articles/exploring-gen-ai/11-multi-file-editing.html
https://martinfowler.com/articles/exploring-gen-ai/13-role-of-developer-
skills.html
https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

系列引用/讨论的一手主源（官方/规范/论文）
OpenAI Harness engineering（官方）
https://openai.com/index/harness-engineering/

GitHub Copilot（官方产品页）
https://github.com/features/copilot

GitHub Copilot multi-file editing（官方变更日志）
https://github.blog/changelog/2024-10-29-multi-file-editing-code-review-custom-
instructions-and-more-for-github-copilot-in-vs-code-october-release-v0-22/

Google Jules（官方博客）
https://blog.google/innovation-and-ai/models-and-research/google-labs/jules/

Model Context Protocol 规范（官方）
https://modelcontextprotocol.io/specification/2025-11-25

AGENTS.md（开放格式官网）
https://agents.md/

Mitchell Hashimoto：My AI Adoption Journey（系列引用）
https://mitchellh.com/writing/my-ai-adoption-journey
```

```
Chad Fowler：Relocating Rigor（系列引用）
https://aicoding.leaflet.pub/3mbrvhyye4k2e

Apple 论文：GSM-Symbolic（系列引用，arXiv HTML 镜像）
https://ar5iv.labs.arxiv.org/html/2410.05229

Chain-of-Thought Prompting 论文（系列引用，arXiv HTML 镜像）
https://ar5iv.labs.arxiv.org/html/2201.11903
```

---

1 58 69 Jules: Google’s autonomous AI coding agent
https://blog.google/innovation-and-ai/models-and-research/google-labs/jules/

2 7 18 82 Exploring Generative AI
https://martinfowler.com/articles/exploring-gen-ai.html

3 51 79 Harness Engineering
https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html

4 27 31 37 64 73 77 To vibe or not to vibe
https://martinfowler.com/articles/exploring-gen-ai/to-vibe-or-not-vibe.html

5 14 35 39 45 75 76 TDD with GitHub Copilot
https://martinfowler.com/articles/exploring-gen-ai/06-tdd-with-coding-assistance.html

6 16 20 36 46 59 60 68 Expanding the solution size with multi-file editing
https://martinfowler.com/articles/exploring-gen-ai/11-multi-file-editing.html

8 21 24 38 52 The role of developer skills in agentic coding
https://martinfowler.com/articles/exploring-gen-ai/13-role-of-developer-skills.html

9 The toolchain
https://martinfowler.com/articles/exploring-gen-ai/01-tool-landscape.html

10 In-line assistance - when is it more useful?
https://martinfowler.com/articles/exploring-gen-ai/03-coding-assistance-when-useful.html

11 12 17 67 81 In-line assistance - how can it get in the way?
https://martinfowler.com/articles/exploring-gen-ai/04-coding-assistance-how-in-the-way.html

13 50 Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl
https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

15 41 57 Coding assistants do not replace pair programming
https://martinfowler.com/articles/exploring-gen-ai/05-not-your-pair-programmer.html

19 Onboarding to a 'legacy' codebase with the help of AI
https://martinfowler.com/articles/exploring-gen-ai/09-ai-help-onboarding-codebase.html

22 66 Median - A tale in three functions
https://martinfowler.com/articles/exploring-gen-ai/02-median-tale.html

23 26 80 Coding Assistants Threaten the Software Supply Chain
https://martinfowler.com/articles/exploring-gen-ai/software-supply-chain-attack-surface.html

25 28 47 Assessing internal quality while coding with an agent
https://martinfowler.com/articles/exploring-gen-ai/ccmenu-quality.html

29 32 40 42 43 48 54 56 65 70 71 72 78 Context Engineering for Coding Agents
https://martinfowler.com/articles/exploring-gen-ai/context-engineering-coding-agents.html

30 33 63 74 Humans and Agents in Software Engineering Loops
https://martinfowler.com/articles/exploring-gen-ai/humans-and-agents.html

34 I still care about the code
https://martinfowler.com/articles/exploring-gen-ai/i-still-care-about-the-code.html

44 Partner with the AI, throw away the code
https://martinfowler.com/articles/exploring-gen-ai/partner-with-ai-and-throw-away-the-code.html

49 What role does LLM reasoning play for software tasks?
https://martinfowler.com/articles/exploring-gen-ai/12-role-of-reasoning.html

53 Harness engineering: leveraging Codex in an agent-first world | OpenAI
https://openai.com/index/harness-engineering/

55 GitHub Copilot · Your AI pair programmer · GitHub
https://github.com/features/copilot

61 Claude Code overview - Claude Code Docs
https://code.claude.com/docs/en/overview?utm_source=chatgpt.com

62 Specification - Model Context Protocol
https://modelcontextprotocol.io/specification/2025-11-25